COMPUTER ENGINEERING

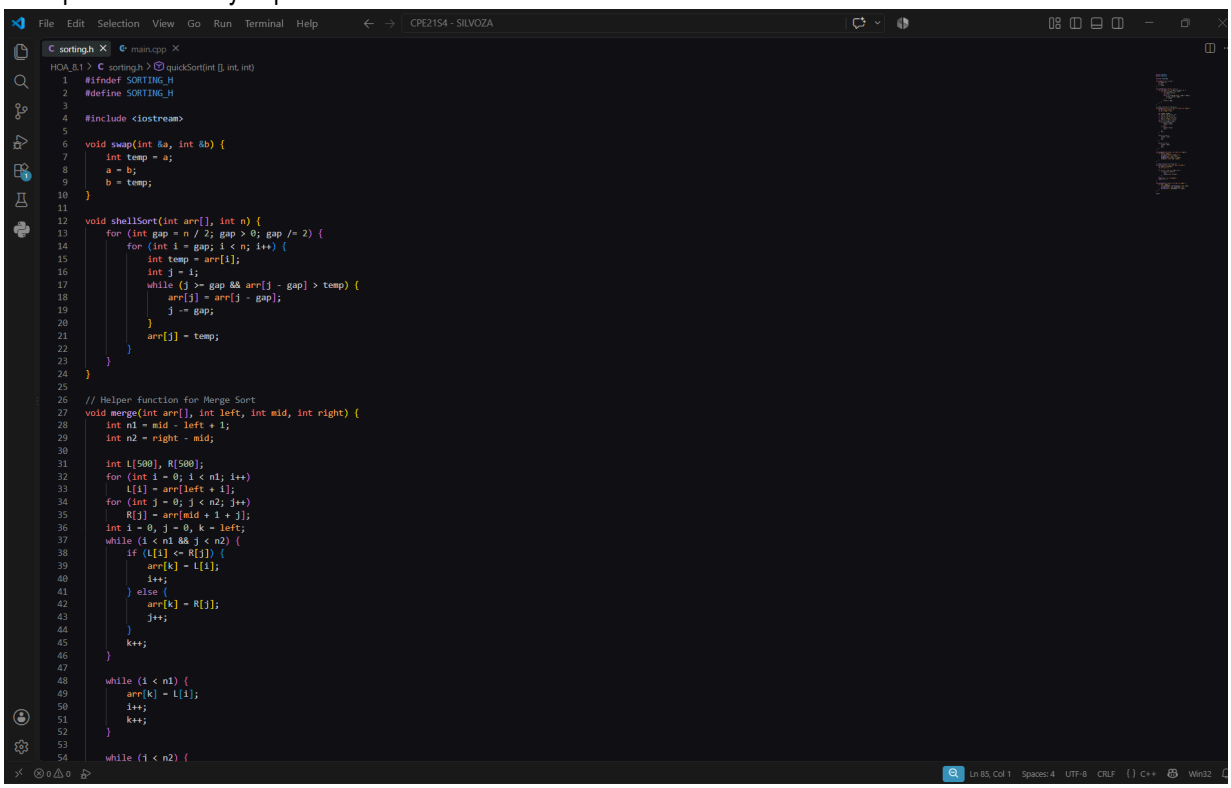
QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	09/15/2026	**DATE SUBMITTED:**	09/15/2026
NAME:	KEITH NICOLE SILVOZA	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. 8.1 SORTING ALGORITHMS PT2

1. PROCEDURE OUTPUT

What procedures did you perform?



```
#ifndef SORTING_H
#define SORTING_H

#include <iostream>

void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

void shellSort(int arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = temp;
        }
    }
}

// Helper function for Merge Sort
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[500], R[500];
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

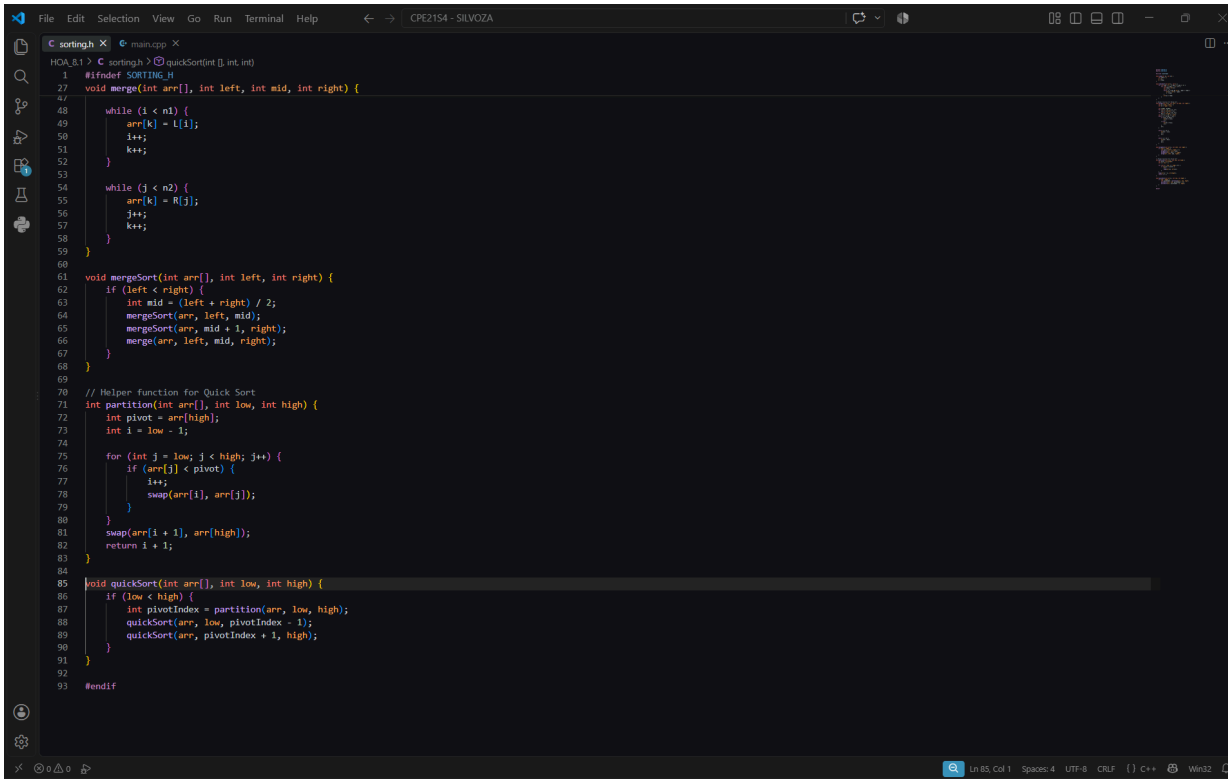
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (i < n2) {
```

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY



```sorting.h
#ifndef SORTING_H
void merge(int arr[], int left, int mid, int right) {

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

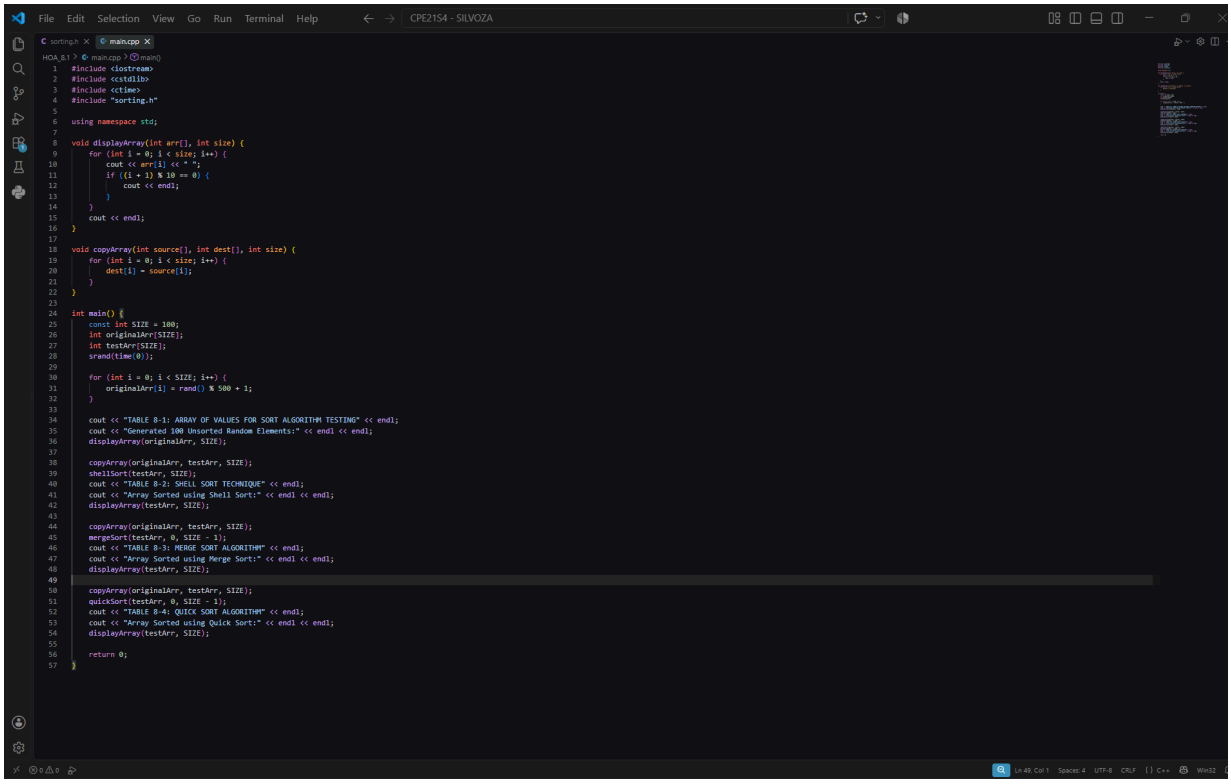
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

// Helper function for Quick Sort
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

#endif
```



```main.cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

using namespace std;

void displayArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
        if ((i + 1) % 10 == 0) {
            cout << endl;
        }
    }
    cout << endl;
}

void copyArray(int source[], int dest[], int size) {
    for (int i = 0; i < size; i++) {
        dest[i] = source[i];
    }
}

int main() {
    const int SIZE = 100;
    int originalArr[SIZE];
    int testArr[SIZE];
    srand(time(0));

    for (int i = 0; i < SIZE; i++) {
        originalArr[i] = rand() % 500 + 1;
    }

    cout << "TABLE 8-1: ARRAY OF VALUES FOR SORT ALGORITHM TESTING" << endl;
    cout << "Generated 100 Unsorted Random Elements:" << endl << endl;
    displayArray(originalArr, SIZE);

    copyArray(originalArr, testArr, SIZE);
    shellSort(testArr, SIZE);
    cout << "TABLE 8-2: SHELL SORT TECHNIQUE" << endl;
    cout << "Array Sorted using Shell Sort:" << endl << endl;
    displayArray(testArr, SIZE);

    copyArray(originalArr, testArr, SIZE);
    mergeSort(testArr, 0, SIZE - 1);
    cout << "TABLE 8-3: MERGE SORT ALGORITHM" << endl;
    cout << "Array Sorted using Merge Sort:" << endl << endl;
    displayArray(testArr, SIZE);

    copyArray(originalArr, testArr, SIZE);
    quickSort(testArr, 0, SIZE - 1);
    cout << "TABLE 8-4: QUICK SORT ALGORITHM" << endl;
    cout << "Array Sorted using Quick Sort:" << endl << endl;
    displayArray(testArr, SIZE);

    return 0;
}
```

CODE	CONSOLE

`cout << "TABLE 8-1: ARRAY OF VALUES FOR SORT ALGORITHM TESTING" << endl;`
`cout << "Generated 100 Unsorted Random Elements:" << endl << endl;`
`displayArray(originalArr, SIZE);`
	TABLE 8-1: ARRAY OF VALUES FOR SORT ALGORITHM TESTING
	Generated 100 Unsorted Random Elements:
	
	109 271 210 404 443 40 175 213 109 475
	9 449 135 275 63 36 353 298 179 414
	27 478 164 420 440 421 176 6 429 211
	383 469 147 198 108 382 460 416 229 259
	129 138 494 214 192 248 476 56 449 8
	432 108 414 428 373 444 159 200 286 431
	141 347 229 467 370 225 6 157 394 293
	84 259 293 343 45 9 97 177 257 31
	350 481 137 142 30 343 167 469 17 21
	333 402 437 130 310 413 171 367 244 168

OBSERVATION: GENERATING 100 RANDOM INTEGERS PRODUCES AN UNSORTED ARRAY. THIS SERVES AS THE INITIAL TEST DATASET FOR EACH SORTING ALGORITHM.

CODE	CONSOLE

`copyArray(originalArr, testArr, SIZE);`
`shellSort(testArr, SIZE);`
`cout << "TABLE 8-2: SHELL SORT TECHNIQUE" << endl;`
`cout << "Array Sorted using Shell Sort:" << endl << endl;`
`displayArray(testArr, SIZE);`
	TABLE 8-2: SHELL SORT TECHNIQUE
	Array Sorted using Shell Sort:
	
	6 6 8 9 9 17 21 27 30 31
	36 40 45 56 63 84 97 108 108 109
	109 129 130 135 137 138 141 142 147 157
	159 164 167 168 171 175 176 177 179 192
	198 200 210 211 213 214 225 229 229 244
	248 257 259 259 271 275 286 293 293 298
	310 333 343 343 347 350 353 367 370 373
	382 383 394 402 404 413 414 414 416 420
	421 428 429 431 432 437 440 443 444 449
	449 460 467 469 469 475 476 478 481 494

OBSERVATION: SHELL SORT ORDERS THE ARRAY BY USING GAP INTERVALS. COMPARING DISTANT ELEMENTS FIRST REDUCES TOTAL MOVEMENTS, LEADING TO A FULLY SORTED ARRAY WHEN THE GAP REACHES 1.

CODE	CONSOLE

`copyArray(originalArr, testArr, SIZE);`
`mergeSort(testArr, 0, SIZE - 1);`
`cout << "TABLE 8-3: MERGE SORT ALGORITHM" << endl;`
`cout << "Array Sorted using Merge Sort:" << endl << endl;`
`displayArray(testArr, SIZE);`
	`TABLE 8-3: MERGE SORT ALGORITHM`
	`Array Sorted using Merge Sort:`
	
	`6 6 8 9 9 17 21 27 30 31`
	`36 40 45 56 63 84 97 108 108 109`
	`109 129 130 135 137 138 141 142 147 157`
	`159 164 167 168 171 175 176 177 179 192`
	`198 200 210 211 213 214 225 229 229 244`
	`248 257 259 259 271 275 286 293 293 298`
	`310 333 343 343 347 350 353 367 370 373`
	`382 383 394 402 404 413 414 414 416 420`
	`421 428 429 431 432 437 440 443 444 449`
	`449 460 467 469 469 475 476 478 481 494`

OBSERVATION: MERGE SORT DIVIDES THE ARRAY INTO SMALLER SUB-ARRAYS RECURSIVELY. RECOMBINING THESE SUB-ARRAYS IN ORDER SUCCESSFULLY SORTS THE ENTIRE LIST IN ASCENDING ORDER.

CODE	CONSOLE

`copyArray(originalArr, testArr, SIZE);`
`quickSort(testArr, 0, SIZE - 1);`
`cout << "TABLE 8-4: QUICK SORT ALGORITHM" << endl;`
`cout << "Array Sorted using Quick Sort:" << endl << endl;`
`displayArray(testArr, SIZE);`
	`TABLE 8-4: QUICK SORT ALGORITHM`
	`Array Sorted using Quick Sort:`
	
	`6 6 8 9 9 17 21 27 30 31`
	`36 40 45 56 63 84 97 108 108 109`
	`109 129 130 135 137 138 141 142 147 157`
	`159 164 167 168 171 175 176 177 179 192`
	`198 200 210 211 213 214 225 229 229 244`
	`248 257 259 259 271 275 286 293 293 298`
	`310 333 343 343 347 350 353 367 370 373`
	`382 383 394 402 404 413 414 414 416 420`
	`421 428 429 431 432 437 440 443 444 449`
	`449 460 467 469 469 475 476 478 481 494`

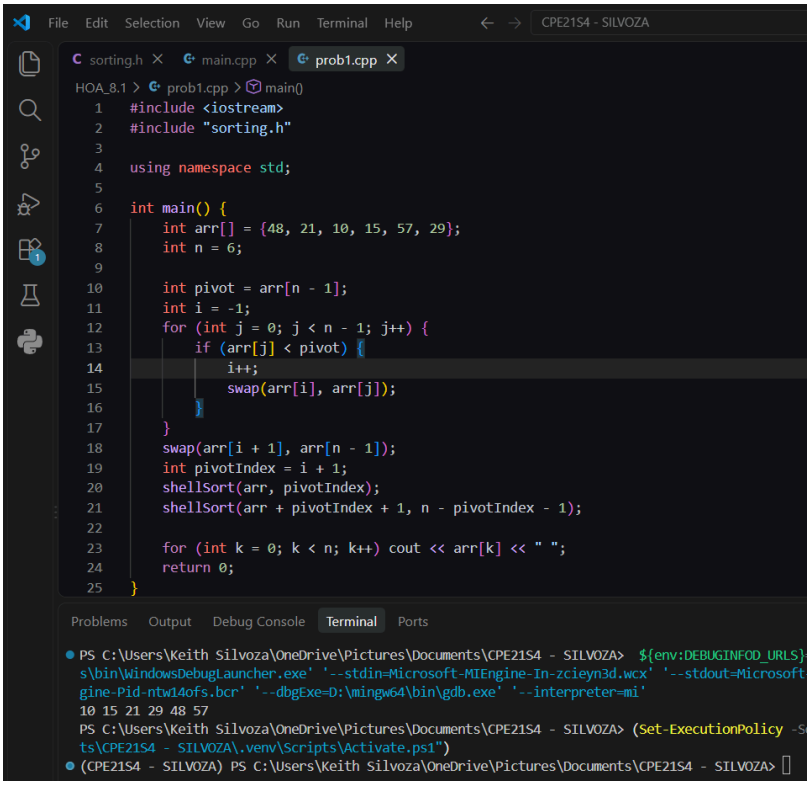
OBSERVATION: QUICK SORT PARTITIONS THE ARRAY USING A PIVOT ELEMENT. REARRANGING ELEMENTS RELATIVE TO THE PIVOT AND SORTING THE SUB-LISTS RECURSIVELY PRODUCES A FULLY ORDERED OUTPUT.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Prob1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example

- Yes, you can sort the left and right sub lists using other algorithms. This is commonly known as a hybrid sort



Prob2:
Quick Sort (or Shell Sort) provides the fastest performance for this specific array because the dataset is very small ($N = 15$). Quick Sort operates directly in-place within the original memory without requiring additional allocations, giving it minimal setup overhead. In contrast, Merge Sort is less efficient for tiny datasets because creating and managing temporary sub-arrays introduces unnecessary processing delay.

Merge Sort and Quick Sort achieve an O(N \log N) time complexity by using a divide-and-conquer approach. The $\log N$ factor represents the total number of levels needed to repeatedly split an array of size N in half until individual elements are reached. At each of these levels, the algorithms process all N elements—Merge Sort compares and merges them, while Quick Sort partitions them around a pivot. Multiplying the N operations performed at each level by the \log N total levels results in an overall time complexity of O(N \log N)

CONCLUSION:

Summary of Lessons Learned

I learned how to implement Shell Sort, Merge Sort, and Quick Sort in C++ using a modular header file. I also understood how gap intervals improve insertion sorting and how divide-and-conquer strategies achieve $O(N \log N)$ efficiency.

Analysis of the Procedure

Generating 100 random integers provided a standard baseline to test all three algorithms on identical data. Copying the array before each test ensured accurate, unbiased outputs, while placing algorithms in sorting.h kept main.cpp clean and organized.

Analysis of the Supplementary Activity

The supplementary tasks showed that Quick Sort can be combined with other algorithms (like Shell Sort) to form hybrid sorts. They also proved that for small datasets ($N = 15$), in-place algorithms outperform Merge Sort by avoiding temporary array allocation.

Concluding Statement & Feedback

I performed well in this activity by successfully writing, compiling, and executing all required C++ code. My main area for improvement is handling dynamic array memory allocation more efficiently during recursive sorting operations.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Copilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, concept explanation, and formatting support.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to help structure C++ code implementations for Shell, Merge, and Quick Sort, as well as clarify time complexity concepts. It did not replace original thinking, analysis, authorship, or completion of the required work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.